Cairo University

Faculty of Engineering

Electronics and Electrical Communications Engineering Department

Third Year

Digital Communications Project I

Name	Section	ID
Serag Khaled Ahmed Abdelaal	2	9221268
Sama Saeed Mohamed Mohamed	2	9220371
Sohaila Ahmed Mohamed Abdelmageed	2	9220377
Manar Kamal Morsi Ahmed	4	9220869
Tarek Osama Mahmoud Eletriby	2	9220414

0.Role of each member

- Sama & Serag: Signal generation and Statistical and time mean
- Sohaila & Manar: Time and Statistical auto correlation
- Tarek: PSD and BW

I. Problem description

The goal of this project is to simulate and analyze digital baseband signaling using different line coding techniques. The focus is on implementing a transmitter that converts random binary data into corresponding voltage waveforms using MATLAB. The transmitted signal, which is random in nature, is considered as a random process that will be analyzed in depth through the project.

II. Introduction

In this project we will send random messages using different line codes, calculating the mean and the statistical and time autocorrelation functions, then evaluating the PSD and the BW for each line code.

III. Control flags:

1) A = 4**:** Amplitude level of bits.
2) NUM_OF_SIGNALS = 500**:** Number of realizations in ensemble.
3) SIGNAL_SIZE = 10**:** The number of bits in each realization.
4) samples_per_bit = 7**:** Each bit will be represented by 7 discrete samples.
5) total_samples = SIGNAL_SIZE * samples_per_bit**:** Total number of samples per realization.

IV. Generation of data

- **Code snippet:**

```
% Define parameters
A = 4;
NUM_OF_SIGNALS = 500;
SIGNAL_SIZE = 100;
samples_per_bit = 7;
total_samples = SIGNAL_SIZE * samples_per_bit;

%Generation of Data
Ensemble_UNRZ = zeros(NUM_OF_SIGNALS, SIGNAL_SIZE*samples_per_bit);
Ensemble_PNRZ = zeros(NUM_OF_SIGNALS, SIGNAL_SIZE*samples_per_bit);
Ensemble_PRZ = zeros(NUM_OF_SIGNALS, SIGNAL_SIZE*samples_per_bit);
```

- **Explanation:**

The first section of code contains:

1. **Parameters definition**

2. Generation of Data:

Initialize three separate matrices each for a different line coding.

V. Creation of unipolar ensemble

- **Code snippet:**

```
for i = 1:NUM_OF_SIGNALS
    bits = randi([0, 1], SIGNAL_SIZE, 1);
    Tx_UNRZ = repelem((bits * A), samples_per_bit);
    Td = randi([0, samples_per_bit - 1]);
    Realization_delayed_UNRZ = circshift(Tx_UNRZ, Td);
    Ensemble_UNRZ(i, :) = Realization_delayed_UNRZ;
```

- **Explanation:**

1. The loop will iterate **NUM_OF_SIGNALS (**Number of realizations in the ensemble), in each iteration a sequence of **100 random bits** is generated using the function **randi .**
2. Then each bit is converted into a corresponding voltage level using this form (**bits * A**): which logic "**1**"is represented by voltage level **A** (which is 4) and logic "**0**"is represented by voltage level **0**.
3. The function **repelem** takes each bit and repeats it 7 times a bit value of **0** is repeated **7 times**, corresponding to activating the DAC for **70 ms** at a voltage of **0 V**. Similarly, a bit value of **1** (which is multiplied by **A = 4**) is also repeated **7 times**, corresponding to activating the DAC for **70 ms** at a voltage of **+A V** this process continues for each bit.
4. After the time shift is applied, the resulting signal is stored in the matrix **Ensemble_UNRZ** at row **i**.

VI. Creation of polar NRZ ensemble

- **Code snippet:**

```
for i = 1:NUM_OF_SIGNALS
    bits = randi([0, 1], SIGNAL_SIZE, 1);
    Tx_PNRZ = repelem(((2 * bits - 1) * A), samples_per_bit);
    Td = randi([0, samples_per_bit - 1]);
    Realization_delayed_PNRZ = circshift(Tx_PNRZ, Td);
    Ensemble_PNRZ(i, :) = Realization_delayed_PNRZ;
```

- **Explanation:**

1. The loop will iterate **NUM_OF_SIGNALS (**Number of realizations in the ensemble), in each iteration a sequence of **100 random bits** is generated using the function **randi().**
2. Then each bit is converted into a corresponding voltage level using this form :(**(2 * bits - 1) * A**) which logic "**1**"is represented by voltage level +**A** (which is +**4**) and logic "**0**"is represented by voltage level -**A** (which is **-4**) .
3. The function **repelem()** takes each bit and repeats it 7 times a bit value of **0** is repeated **7 times**, corresponding to activating the DAC for **70 ms** at a voltage of **0 V**. Similarly, a bit value of **1** (which is multiplied by **A = 4**) is also repeated **7 times**, corresponding to activating the DAC for **70 ms** at a voltage of **+A V** this process continues for each bit.
4. After the time shift is applied, the resulting signal is stored in the matrix **Ensemble_PNRZ** at row **i**.

VII. Creation of polar RZ ensemble

- **Code snippet:**

```
for i = 1:NUM_OF_SIGNALS
    bits = randi([0, 1], SIGNAL_SIZE, 1);
    Tx_RZ = zeros(total_samples, 1);

        for j = 1:SIGNAL_SIZE
        start = (j - 1) * samples_per_bit + 1;
        End = start + samples_per_bit - 1;

        half_time = floor(samples_per_bit / 2);

        if bits(j) == 1
            Tx_RZ(start:start + half_time - 1) = A;
            Tx_RZ(start + half_time:End) = 0;
        else
            Tx_RZ(start:start + half_time - 1) = -A;
            Tx_RZ(start + half_time:End) = 0;
        end
       end

    Td = randi([0, samples_per_bit - 1]);
    Realization_delayed_PRZ = circshift(Tx_RZ, Td);
    Ensemble_PRZ(i, :) = Realization_delayed_PRZ;
```

- **Explanation:**

1. The loop will iterate **NUM_OF_SIGNALS (**Number of realizations in the ensemble), in each iteration a sequence of **100 random bits** is generated using the function **randi.**
2. Each bit is assigned a time duration of **samples_per_bit** samples. This time is divided into two equal halves:

1) **First Half** is from **start** of the bit to (**(start +floor(samples_per_bit / 2)) - 1**)

   - If the bit is 1, the first half is set to a positive voltage (A).

   - If the bit is 0, the first half is set to a negative voltage (-A).

2) **Second Half:**The second half of the time is always set to zero, representing the "**return-to-zero**" part of the Polar RZ signal.
3) After the time shift is applied, the resulting signal is stored in the matrix **Ensemble_PNRZ** at row **i**.

VIII. Applying random initial time shifts for each waveform

- **Code snippet:**

Unipolar NRZ	`Tx_UNRZ = repelem((bits * A), samples_per_bit);`
	`% Apply the delay`
	`Td = randi([0, samples_per_bit - 1]);`
	`Realization_delayed_UNRZ = circshift(Tx_UNRZ, Td);`
	`Ensemble_UNRZ(i, :) = Realization_delayed_UNRZ;`

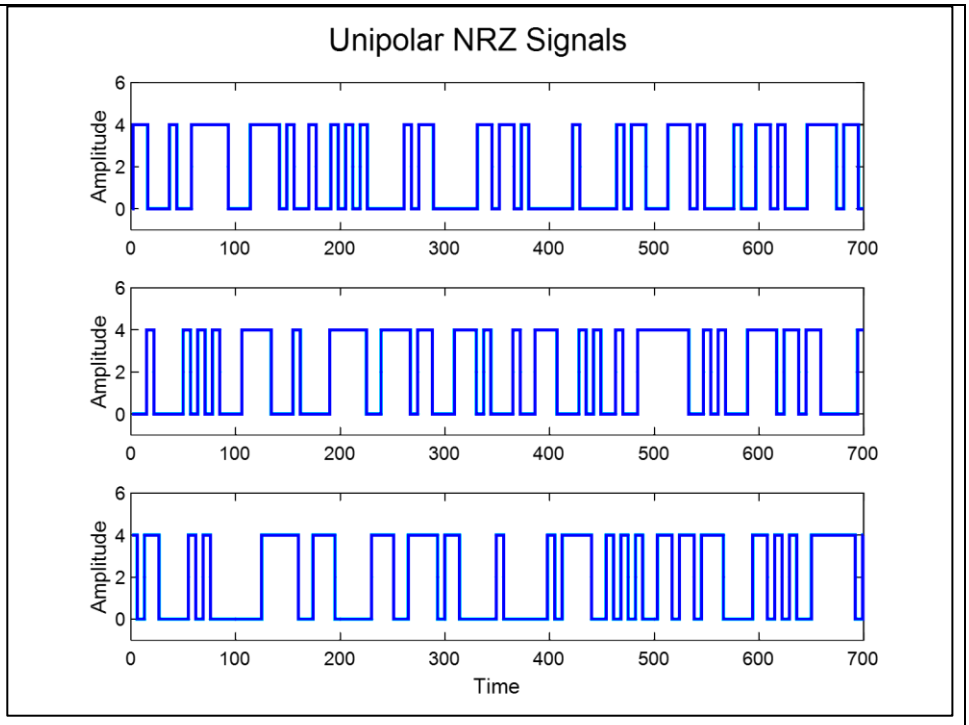
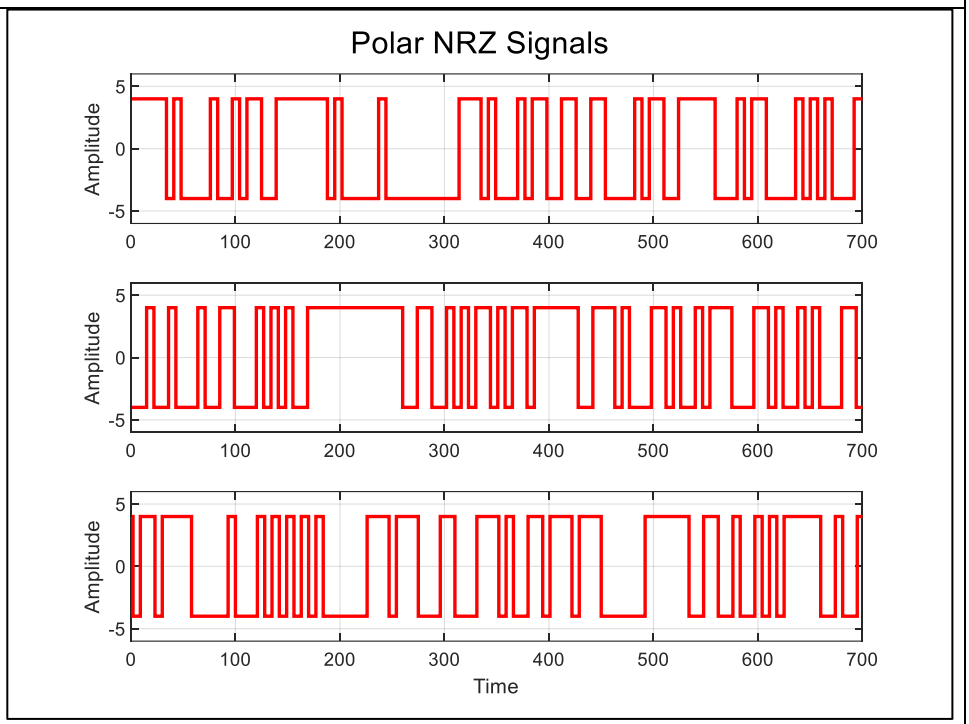
Polar NRZ	`Tx_PNRZ = repelem(((2 * bits - 1) * A), samples_per_bit);`
	`% Apply the delay`
	`Td = randi([0, samples_per_bit - 1]);`
	`Realization_delayed_PNRZ = circshift(Tx_PNRZ, Td);`
	`Ensemble_PNRZ(i, :) = Realization_delayed_PNRZ;`

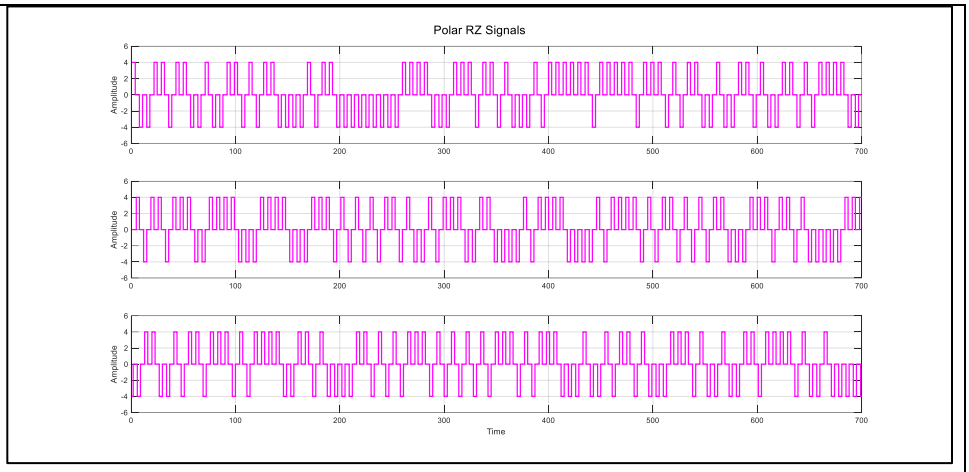
Polar RZ	`% Apply the delay`
	`Td = randi([0, samples_per_bit - 1]);`
	`Realization_delayed_PRZ = circshift(Tx_RZ, Td);`
	`Ensemble_PRZ(i, :) = Realization_delayed_PRZ;`

- **Explanation:**

A random delay **Td** is generated using **randi([0, samples_per_bit - 1])**, which gives a random integer between **0** and **samples_per_bit - 1**. which means the signal will be shifted by a random amount between 0 and 6 sample positions.This delay is applied to the signal using the **circshift** function, which shifts the signal by **Td** samples.

- **Plotting:**

Unipolar NRZ	
Polar NRZ	

Polar RZ	

IX. Getting the cell arrays ready to calculate the statistical mean and autocorrelation

1. Unipolar NRZ

- **Code snippet:**

```
Mean_UNRZ = zeros(1, SIGNAL_SIZE);
Sum_UNRZ = 0;
Time_Sum_UNRZ = 0;
for i = 1:total_samples
    for j = 1 : NUM_OF_SIGNALS
        Sum_UNRZ = Sum_UNRZ + Ensemble_UNRZ(j,i);
    end
    Mean_UNRZ(i) = Sum_UNRZ/NUM_OF_SIGNALS;
    Sum_UNRZ = 0;
end
```

- **Explanation:**

  It is as simple as any mean calculation; a summation of all values then division by the number of values. To do that, Mean_UNRZ is assigned all zeros at the beginning so that we reset any residual values from a past run of the program. Then we calculate the summation of the bits across the ensemble. Then we divide by the number of signals too get the 1st mean value, and repeat across time.

- **Hand Analysis**

P(X=A) = P(X=0) = 0.5

A=4

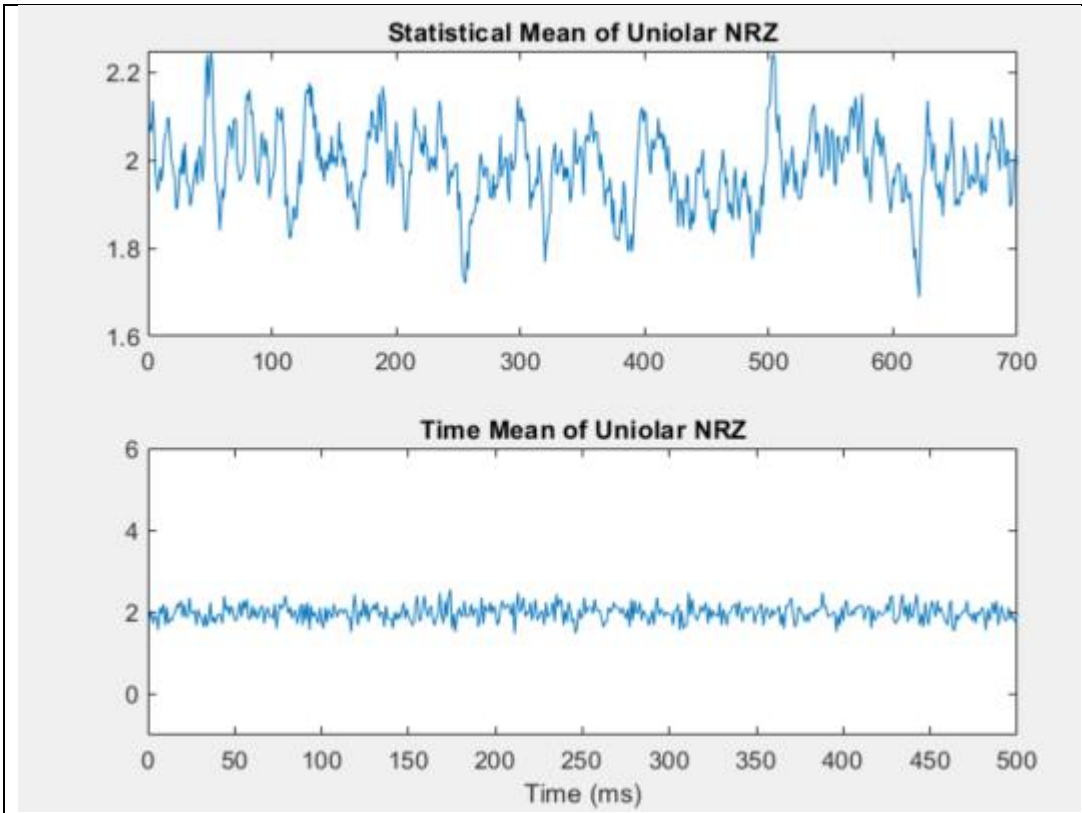
Random variables at each time instant are identical distribution [f(X(t1)) = f(X(t2))=……]

X(t)	**A**	**0**
f(X(t))	**0.5**	**0.5**

mean of random process X(t) at any time instant = mean of random variable at this instant

∴ E(X(t)) = (A*P(X=A)) + (0*P(X=0)) = A*0.5 + 0*0.5 = A/2 = 2

- **Plotting**



➢ **Comment:** The graph shows the mean of the ensemble across time.As expected, the statistical mean of the Unipolar NRZ signal remains relatively constant over time, fluctuating slightly around the average value of approximately 2.

2. Polar NRZ

- Code snippet:

```
Mean_PNRZ = zeros(1, SIGNAL_SIZE);
Sum_PNRZ = 0;
Time_Sum_PNRZ = 0;
for i = 1:total_samples
    for j = 1 : NUM_OF_SIGNALS
        Sum_PNRZ = Sum_PNRZ + Ensemble_PNRZ(j,i);
    end
    Mean_PNRZ(i) = Sum_PNRZ/NUM_OF_SIGNALS;
    Sum_PNRZ = 0;
end
```

- **Explanation:**

  It is as simple as any mean calculation; a summation of all values then division by the number of values. To do that, Mean_PNRZ is assigned all zeros at the beginning so that we reset any residual values from a past run of the program. Then we calculate the summation of the bits across the ensemble. Then we divide by the number of signals too get the 1st mean value, and repeat across time.

- **Hand Analysis**

P(X=+A) = P(X=-A) = 0.5

A=4

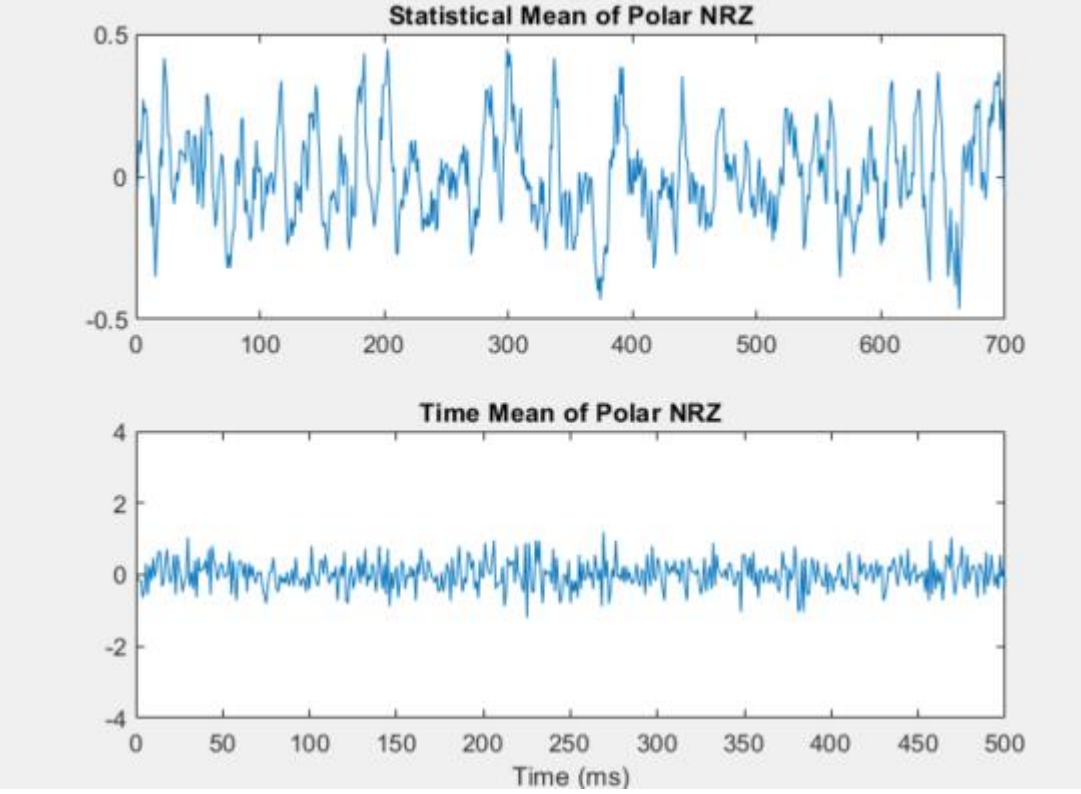
Random variables at each time instant are identical distribution [f(X(t1)) = f(X(t2))=……]

X(t)	**+A**	**-A**
f(X(t))	**0.5**	**0.5**

mean of random process X(t) at any time instant = mean of random variable at this instant

∴ E(X(t)) = (+A*P(X=+A)) + (-A*P(X=-A)) = +A*0.5 + -A*0.5 = 0

- **Plotting**



- **Comment:** The graph shows the mean of the ensemble across time.As expected, the statistical mean of the polar NRZ signal remains relatively constant over time, fluctuating slightly around the average value of approximately 0.

3. Polar RZ

- **Code snippet :**

```
Mean_PRZ = zeros(1, SIGNAL_SIZE);

Sum_PRZ = 0;
Time_Sum_PRZ = 0;
for i = 1:total_samples
    for j = 1 : NUM_OF_SIGNALS
        Sum_PRZ = Sum_PRZ + Ensemble_PRZ(j,i);
    end
    Mean_PRZ(i) = Sum_PRZ/NUM_OF_SIGNALS;

    Sum_PRZ = 0;
end
```

- **Explanation:**

  It is as simple as any mean calculation; a summation of all values then division by the number of values. To do that, Mean_PNRZ is assigned all zeros at the beginning so that we reset any residual values from a past run of the program. Then we calculate the summation of the bits across the ensemble. Then we divide by the number of signals too get the 1st mean value, and repeat across time.

- **Hand Analysis**

P(X=+A) = P(X=-A) =1/4

P(X=0) = 1/2

A=4

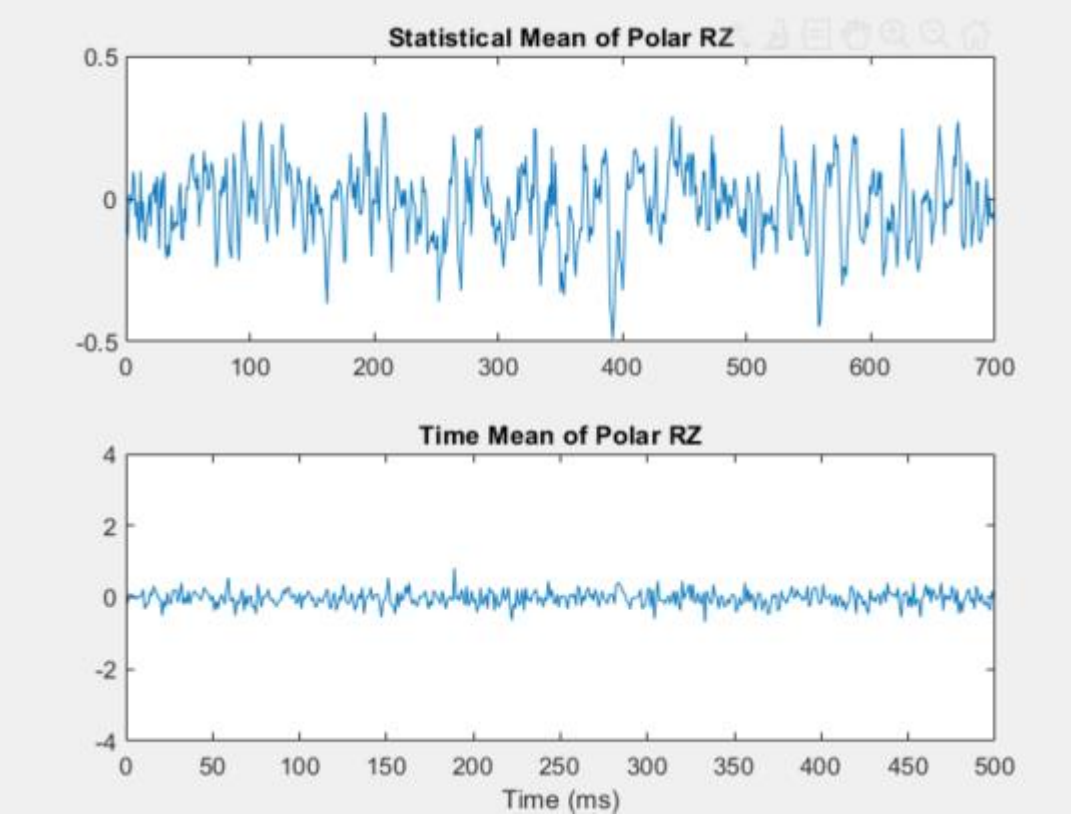
Random variables at each time instant are identical distribution [f(X(t1)) = f(X(t2))=……]

X(t)	+A	-A	0
f(X(t))	1/4	1/4	1/2

mean of random process X(t) at any time instant = mean of random variable at this instant

∴ E(X(t)) = (+A*P(X=+A)) + (-A*P(X=-A)) + (0*P(X=0) = +A*1/4 + -A*1/4 + 0*1/2 = 0

- Plotting



- **Comment** : The graph shows the mean of the ensemble across time.As expected, the statistical mean of the polar RZ signal remains relatively constant over time, fluctuating slightly around the average value of approximately 0.

- **Results:**

Line code	UniPolar NRZ	Polar NRZ	Polar RZ
Theoretical	**2**	**0**	**0**
Stimulated	Between **2.048** and **2.216**	Between **0.032** and **0.192**	Between **0.04** and **0.088**

XII. Q3: Calculating the autocorrelation

1. *Statistical autocorrelation*

$$\mathcal{R}_x(\tau) = E\{x(t) \cdot x(t+\tau)\}$$ [1]

The code for the calculation of the statistical autocorrelation for the **Unipolar NRZ** signal is shown:

```
%% ======= Statistical Autocorrelation For UNRZ ========
Stat_AutoCorr_UNRZ = zeros(1, total_samples);

sample = 1;
        for tau = 0:total_samples-1
            sample_values1 = Ensemble_UNRZ(:, sample);
            sample_values2 = Ensemble_UNRZ(:, sample+tau);
            squared_values = sample_values1 .* sample_values2;
            Stat_AutoCorr_UNRZ(tau + 1) = sum(squared_values) /
NUM_OF_SIGNALS;
        end
```

```
% Make it symmetric
Stat_AutoCorr_UNRZ_Flipped = fliplr(Stat_AutoCorr_UNRZ(2:end));
Stat_AutoCorr_Even = [Stat_AutoCorr_UNRZ_Flipped, Stat_AutoCorr_UNRZ];
```

Table 1: Statistical autocorrelation code

Explanation:

To calculate the **statistical** autocorrelation, we **loop** over the ensemble, and for each realization, the signal is **circularly shifted** by τ samples using *circshift*. The original signal is **multiplied element-wise** by its shifted version. The resulting products are **summed** across all time samples, and the **average** is taken over both the **realizations** and **time samples** to compute the expected value $E[x(t)\cdot x(t+\tau)]$. Since autocorrelation is **symmetric**, the output is **mirrored** to include both negative and positive lags.

Calculating theoretical statistical autocorrelation:

```
% ======= Theoretical Statistical Autocorrelation ========

Tb_theory = samples_per_bit;  % Rename Tb for theoretical calculations
tau_theory_values = 0:(total_samples - 1);  % Consider only non-negative
lags

right_half_theory = zeros(1, total_samples);

for i = 1:total_samples
    tau_theory = tau_theory_values(i);  % Use the theoretical tau values
    if tau_theory >= Tb_theory
        right_half_theory(i) = A^2 / 4;
    else
        right_half_theory(i) = (A^2 / 2) * (1 - 0.5 * (tau_theory /
Tb_theory));
    end
end

% Mirror the right half to get the full theoretical autocorrelation
Theoretical_AutoCorr         =         [fliplr(right_half_theory(2:end)),
right_half_theory];
```

Table 2: Theoretical statistical autocorrelation code

Derivation of statistical autocorrelation:

1. Unipolar NRZ

- P(x=A) =p(x=-A) = 0.5
- [A , -A] = [4 , 0]

Case 1: $\tau < T_b$

$R_x(\tau) = E(x(t).x(t+\tau)) = A^2 * p(A,A) + 2*A*0*p(A,0) + 0^2 * p(0,0) = A^2 * p(A,A)$

$p(A,A) = p(x(t+\tau) = A \mid x(t) = A) * p(A) = 0.5*p(x(t+\tau) = A \mid x(t) = A)$

$p(x(t+\tau) = A \mid x(t) = A) = p(\text{no transition}) + 0.5*p(\text{transition})$

Due to the uniform distribution of a bit:

$p(\text{transition}) = \int_{t1}^{t2} \frac{1}{Tb} = \frac{\tau}{Tb}$ $\quad p(\text{no transition}) = 1 - \frac{\tau}{Tb}$

$\therefore p(x(t+\tau) = A \mid x(t) = A) = (1 - \frac{\tau}{Tb}) + 0.5*\frac{\tau}{Tb} = 1 - 0.5\frac{\tau}{Tb}$

$\therefore p(A,A) = \frac{1}{2}(1 - 0.5\frac{\tau}{Tb})$

$\therefore \mathbf{R_x(\tau) = \frac{A2}{2} * (1 - 0.5\frac{\tau}{Tb})}$

Case 2: $\tau > T_b$

since all samples are independent

$\therefore R_x(\tau) = A^{2*} p(A,A) = A^2*0.5*0.5$

$\therefore R_x(\tau) = \frac{1}{4}A^2$

$$\therefore R_x(\tau) = \begin{cases} \frac{A^2}{2}\left(1 - \frac{\tau}{2T_b}\right) & \tau < T_b \\ \frac{1}{4}A^2 & \tau > T_b \end{cases}$$

Total power = $R_x(0) = \frac{16}{2} = 8$

DC power = $R_x(\infty) = \frac{16}{4} = 4$

AC power = total power – DC power = 8 – 4 = 4

2. Polar NRZ

Using the previous derivations:

- P(x=A) =p(x=-A) = 0.5
- [A , -A] = [4 , -4]
- p(A,A) = p(-A,-A), p(-A,A) = p(A,-A)

$R_x(\tau) = E(x(t).x(t+\tau)) = A^{2*} p(A,A) + A*(-A)*p(A,-A) + (-A)*A*p(-AA) + (-A)^2*p(-A,-A)$
$= 2A^{2*} p(A,A) - 2A^2*p(A,-A)$

Case 1: $\tau < T_b$

p(no transition) = $1 - \frac{\tau}{Tb}$, p(transition) = $\frac{\tau}{Tb}$

p(A,A) = $\frac{1}{2}\left(1 - 0.5\frac{\tau}{Tb}\right)$

p(A,-A) = 0.5*p(transition) = $0.5 * 0.5 * \frac{\tau}{Tb} = 0.25 * \frac{\tau}{Tb}$

$\therefore R_x(\tau) = 2A^2\left(\frac{1}{2}\left(1 - 0.5\frac{\tau}{Tb}\right)\right) - 2A^2(0.25 * \frac{\tau}{Tb})$

$\therefore R_x(\tau) = A^2\left(1 - \frac{\tau}{Tb}\right)$

Case 2: $\tau > T_b$

There will always be a transition so:

$R_x(\tau) = E(x(t).x(t+\tau)) = 2A^{2*} p(A,A) - 2A^2*p(A,-A) = A^2*p(transition) - A^2*p(transition)$

$R_x(\tau) = 0$

$$\therefore Rx(\tau) = \begin{cases} A^2\left(1 - \frac{\tau}{T_b}\right) & \tau < T_b \\ 0 & \tau < T_b \end{cases}$$

Total power = $R_x(0) = 16$

DC power = $R_x(\infty) = 0$

AC power = total power – DC power = 16 – 0 = 16

3. Polar RZ

Using the above derivations and steps:

$$\therefore Rx(\tau) = \begin{cases} \frac{A^2}{2}\left(1 - \frac{2\tau}{T_b}\right) & \tau < T_b \\ 0 & \tau < T_b \end{cases}$$

Total power = $R_x(0) = \frac{16}{2} = 8$

DC power = $R_x(\infty) = 0$

AC power = total power – DC power = 8 – 0 = 8

Line codes\Power	**Total Power $\mathcal{R}_x(0)$**	**DC Power $\mathcal{R}_x(\infty)$**	**AC Power $\mathcal{R}_x(0) - \mathcal{R}_x(\infty)$**
Unipolar RZ	8	4	4
Polar NRZ	16	0	16

Polar RZ	8	0	8

Table 3: Power for each line code

2. Time autocorrelation

$$\mathcal{R}_x(\tau) = \frac{1}{N}\sum_n x[n] \cdot x[n+k]$$ [1]

The code for the calculation of the time autocorrelation for the **Unipolar NRZ** signal is shown:

```
%% ======= Time Autocorrelation For UNRZ ========
x_t = Ensemble_UNRZ(1, :);
Time_AutoCorr_UNRZ = zeros(1, total_samples);

for tau = 0:total_samples-1
    shifted_x = circshift(x_t, [0, tau]);
    sum_corr = sum(x_t .* shifted_x);

    % Average over time
    Time_AutoCorr_UNRZ(tau + 1) = sum_corr / total_samples;
end

% Make it symmetric
Time_AutoCorr_UNRZ_Flipped = fliplr(Time_AutoCorr_UNRZ(2:end));
Time_AutoCorr_Even = [Time_AutoCorr_UNRZ_Flipped, Time_AutoCorr_UNRZ];
```

Table 4: Time autocorrelation code

To compute the **time** autocorrelation, **one realization** from the ensemble is selected. For each time lag τ, the signal is **circularly shifted** using *circshift*, then **multiplied element-wise** with the original signal. The result is **summed** across time and **averaged** by the **number of samples**. Finally, since autocorrelation is an **even** function, the result is **mirrored** to represent both negative and positive lags.

XIII. Plotting the autocorrelation

1. Statistical autocorrelation

```
% Define lag axis
tau_values = - (total_samples - 1) : (total_samples - 1);

% Define theoretical tau axis
tau_theory_full = [-fliplr(tau_theory_values(2:end)), tau_theory_values];

% Plot Statistical and Theoretical Autocorrelation
figure;
subplot(3,1,1);
plot(tau_values, Stat_AutoCorr_Even, 'r', 'LineWidth', 1.5); hold on;
plot(tau_theory_full, Theoretical_AutoCorr, 'b--', 'LineWidth', 1.5);
legend('Simulated', 'Theoretical');
title('Statistical Autocorrelation of Unipolar NRZ');
xlabel('Lag (\tau)');
ylabel('Autocorrelation');
grid on;
xlim([-30 30]);  % Adjust x-axis range
```

Table 5: Statistical autocorrelation plotting code

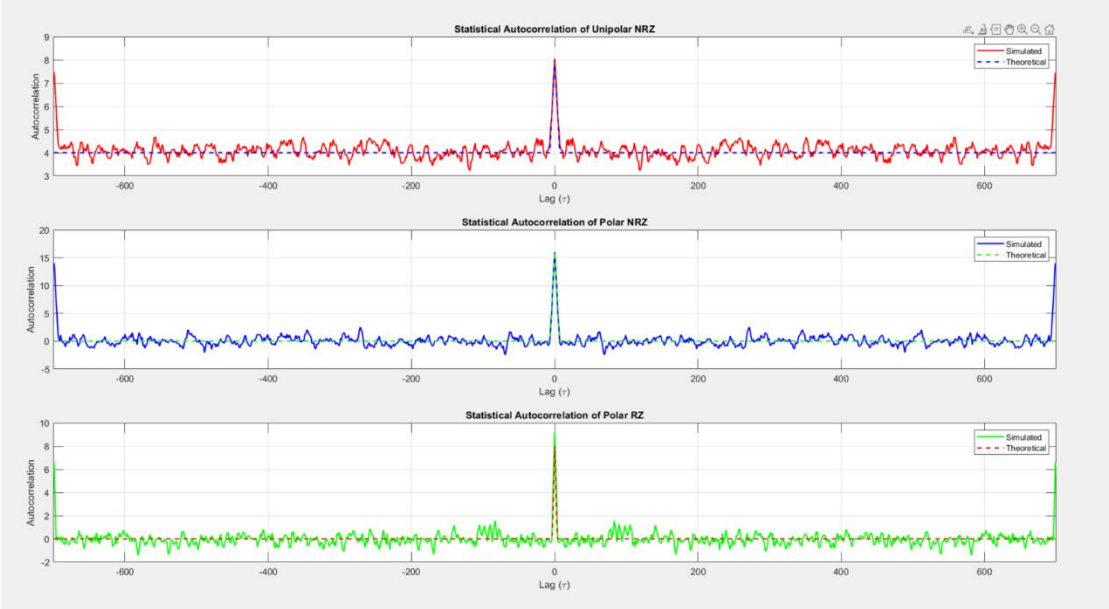


Figure 1: Statistical autocorrelation plots

The statistical and theoretical autocorrelation functions of the **Unipolar NRZ** signal are plotted against the lag axis. The simulated results are shown in red, while the theoretical values are shown as a dashed blue line.

2. Time autocorrelation

```
% Plot Time Autocorrelation
figure;
subplot(3,1,1);
plot(tau_values, Time_AutoCorr_Even,'r', 'LineWidth', 1.5);
title('Time Autocorrelation of Unipolar NRZ');
xlabel('Lag (\tau)');
ylabel('Autocorrelation');
grid on;
xlim([-30 30]);  % Adjust x-axis range
```

Table 6: Time autocorrelation plotting code

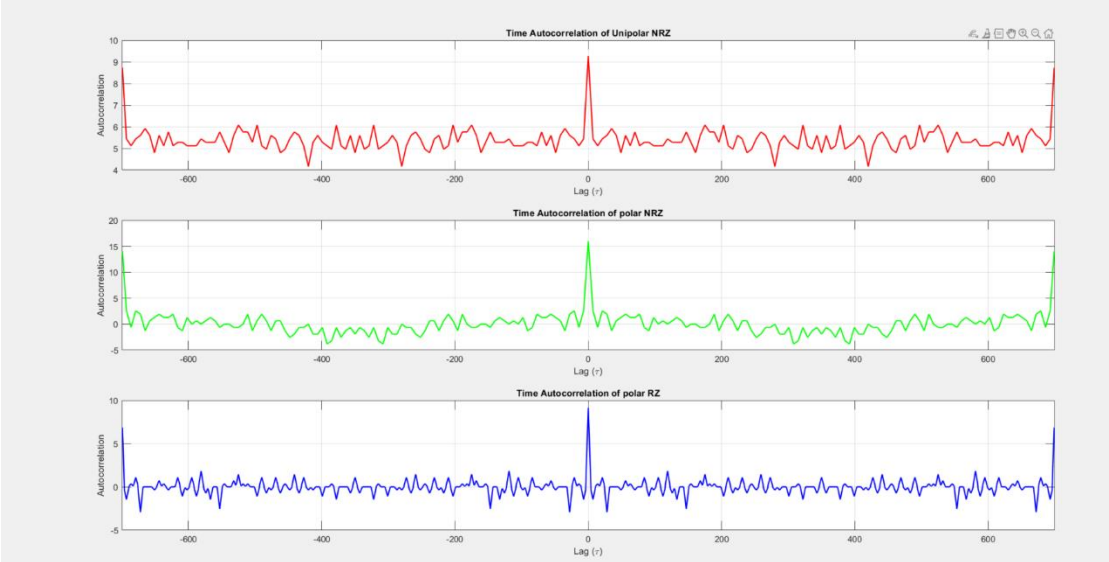


Figure 2: Time autocorrelation plots

Observing the two graphs, we can see that the statistical autocorrelation is almost identical to the time autocorrelation. This indicates that this process is *ergodic*.

XIIII. Q2: Is the process stationary?

- *Mean:* From the graph, we can see that the mean remains constant across time.
- *Autocorrelation:* The autocorrelation function depends only on the time lag (τ) and not on the specific time instant.

Therefore, the process satisfies the conditions of **wide-sense stationarity (WSS)**.

XV. Q4: Computing the time mean and autocorrelation of one waveform

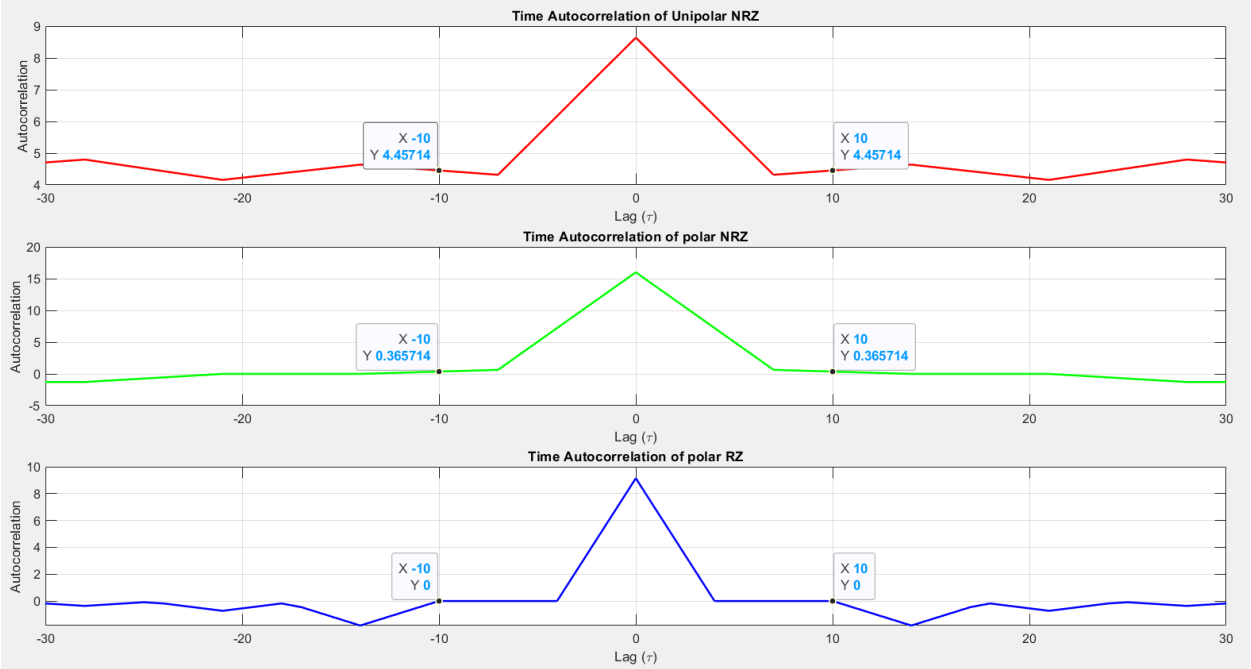


Figure 3: Time autocorrelation for one waveform

XVI. Q5 Is the random process ergodic?

- *Mean:* The time mean is equal to the statistical mean.
- *Autocorrelation:* The time autocorrelation is equal to the statistical autocorrelation.

Since the time averages are equal to the ensemble averages, the process can be considered **ergodic.** [1]

XVII. Plotting the PSD

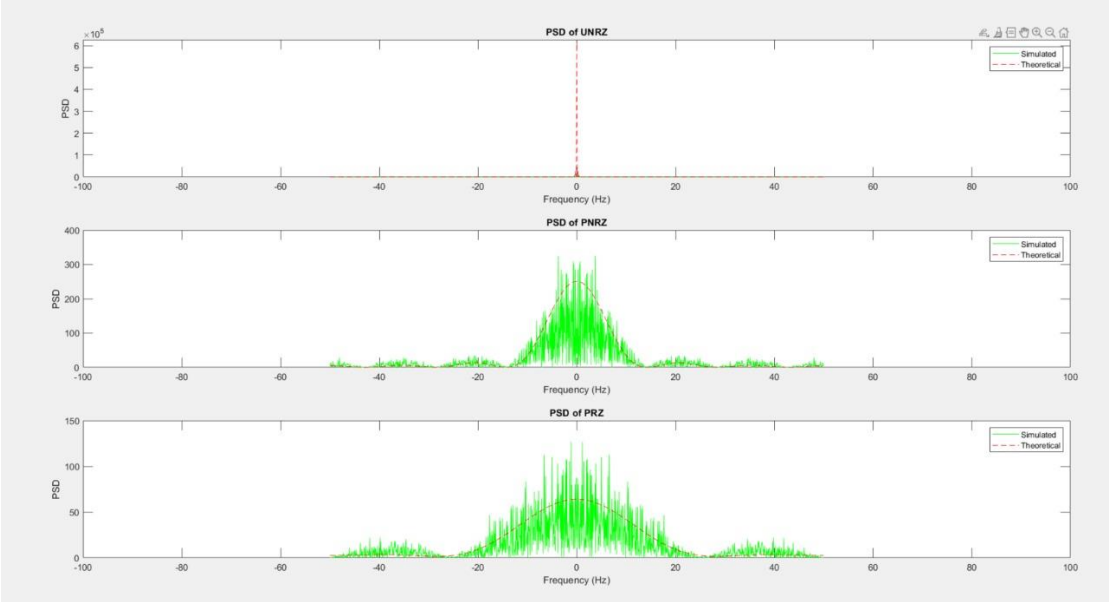


Figure 4 PSD for different line codes

XVX. the bandwidth of the transmitted signal

This is extracted from the theoretical PSD graphs

- **UNRZ BW= 0.1Hz (This ideally should be around 10 Hz however ther must have been an error)**
- **PNRZ BW= 14 Hz**
- **PRZ BW= 25Hz**

Appendix

```
% Define parameters
A = 4;
SIGNAL_SIZE = 100;
NUM_OF_SIGNALS = 500;
samples_per_bit = 7;
total_samples = SIGNAL_SIZE * samples_per_bit;
fs=100;
Ensemble_UNRZ = zeros(NUM_OF_SIGNALS, SIGNAL_SIZE*samples_per_bit);
Ensemble_PNRZ = zeros(NUM_OF_SIGNALS, SIGNAL_SIZE*samples_per_bit);
Ensemble_PRZ = zeros(NUM_OF_SIGNALS, SIGNAL_SIZE*samples_per_bit);

%% ================= Unipolar NRZ ==================
% figure;
% sgtitle('Unipolar NRZ Signals');

for i = 1:NUM_OF_SIGNALS
    bits = randi([0, 1], SIGNAL_SIZE, 1);
```

```
    Tx_UNRZ = repelem((bits * A), samples_per_bit);
    Td = randi([0, samples_per_bit - 1]);
    Realization_delayed_UNRZ = circshift(Tx_UNRZ, Td);
    Ensemble_UNRZ(i, :) = Realization_delayed_UNRZ;

%     subplot(NUM_OF_SIGNALS, 1, i);
%     stairs(Realization_delayed_UNRZ, 'LineWidth', 1.5);
%     ylabel(sprintf('Realization %d', i));
%     ylim([-1, A + 2]);

%     grid on;
end
%xlabel('Time');

%Statistical and Time mean Calculations
Mean_UNRZ = zeros(1, SIGNAL_SIZE);

Sum_UNRZ = 0;


for i = 1:SIGNAL_SIZE*7
    for j = 1 : NUM_OF_SIGNALS
        Sum_UNRZ = Sum_UNRZ + Ensemble_UNRZ(j,i);
    end
    Mean_UNRZ(i) = Sum_UNRZ/NUM_OF_SIGNALS;
    Sum_UNRZ = 0;
end

sum_time = zeros(NUM_OF_SIGNALS, 1);
for i = 1:NUM_OF_SIGNALS
    for j = 1:total_samples
        sum_time(i) = sum_time(i) + Ensemble_UNRZ(i, j);
    end
    Time_Mean_UNRZ = sum_time / total_samples;
end
figure;
subplot(2,1,1);
plot(1:length(Mean_UNRZ), Mean_UNRZ);
title('Statistical Mean of Uniolar NRZ');
subplot(2,1,2);
plot(1:length(Time_Mean_UNRZ), Time_Mean_UNRZ);
ylim([-1, A + 2]);
title('Time Mean of Uniolar NRZ');

% Adjust figure layout
xlabel('Time (ms)');
%% ================= Polar NRZ ==================

% sgtitle('Polar NRZ Signals');

for i = 1:NUM_OF_SIGNALS
    bits = randi([0, 1], SIGNAL_SIZE, 1);
    Tx_PNRZ = repelem(((2 * bits - 1) * A), samples_per_bit);
    Td = randi([0, samples_per_bit - 1]);
    Realization_delayed_PNRZ = circshift(Tx_PNRZ, Td);
    Ensemble_PNRZ(i, :) = Realization_delayed_PNRZ;

%     subplot(NUM_OF_SIGNALS, 1, i);
%     stairs(Realization_delayed_PNRZ, 'LineWidth', 1.5);
```

```
%     ylabel(sprintf('Realization %d', i));
%     ylim([-A - 2, A + 2]);
%     grid on;
end
% xlabel('Time');

% Statistical and Time Mean Calculations

Mean_PNRZ = zeros(1, SIGNAL_SIZE);

Sum_PNRZ = 0;
Time_Sum_PNRZ = 0;
for i = 1:SIGNAL_SIZE*7
    for j = 1 : NUM_OF_SIGNALS
        Sum_PNRZ = Sum_PNRZ + Ensemble_PNRZ(j,i);
    end
    Mean_PNRZ(i) = Sum_PNRZ/NUM_OF_SIGNALS;
    Sum_PNRZ = 0;
end

sum_time = zeros(NUM_OF_SIGNALS, 1);
for i = 1:NUM_OF_SIGNALS
    for j = 1:total_samples
        sum_time(i) = sum_time(i) + Ensemble_PNRZ(i, j);
    end
    Time_Mean_PNRZ = sum_time / total_samples;
end
figure;
subplot(2,1,1);
plot(1:length(Mean_PNRZ), Mean_PNRZ);
title('Statistical Mean of Polar NRZ');
subplot(2,1,2);
plot(1:length(Time_Mean_PNRZ), Time_Mean_PNRZ);
ylim([-A, A]);
title('Time Mean of Polar NRZ');

% Adjust figure layout
xlabel('Time (ms)');


%% ================= Polar RZ ==================

% sgtitle('Polar RZ Signals');

for i = 1:NUM_OF_SIGNALS
    bits = randi([0, 1], SIGNAL_SIZE, 1);
    Tx_RZ = zeros(total_samples, 1);

    for j = 1:SIGNAL_SIZE
        start = (j - 1) * samples_per_bit + 1;
        End = start +3;

        if bits(j) == 1
            Tx_RZ(start:End) = A;
        else
            Tx_RZ(start:End) = -A;
        end
    end
```

```
    Td = randi([0, samples_per_bit - 1]);
    Realization_delayed_PRZ = circshift(Tx_RZ, Td);
    Ensemble_PRZ(i, :) = Realization_delayed_PRZ;

%     subplot(NUM_OF_SIGNALS, 1, i);
%     stairs(Realization_delayed_PRZ, 'LineWidth', 1.5);
%     ylabel(sprintf('Realization %d', i));
%     ylim([-A - 2, A + 2]);
%     grid on;
end
%xlabel('Time');

% Statistical and Time Mean Calculations
Mean_PRZ = zeros(1, SIGNAL_SIZE);

Sum_PRZ = 0;
Time_Sum_PRZ = 0;

for i = 1:SIGNAL_SIZE*7
    for j = 1 : NUM_OF_SIGNALS
        Sum_PRZ = Sum_PRZ + Ensemble_PRZ(j,i);
    end
    Mean_PRZ(i) = Sum_PRZ/NUM_OF_SIGNALS;
    Sum_PRZ = 0;
end
sum_time = zeros(NUM_OF_SIGNALS, 1);
for i = 1:NUM_OF_SIGNALS
    for j = 1:total_samples
        sum_time(i) = sum_time(i) + Ensemble_PRZ(i, j);
    end
    Time_Mean_PRZ = sum_time / total_samples;
end

figure
subplot(2,1,1);
plot(1:length(Mean_PRZ), Mean_PRZ);
title('Statistical Mean of Polar RZ');
subplot(2,1,2);
plot(1:length(Time_Mean_PRZ), Time_Mean_PRZ);
ylim([-A, A]);
title('Time Mean of Polar RZ');

% Adjust figure layout
xlabel('Time (ms)');

%% ======= Statistical Autocorrelation For UNRZ ========
Stat_AutoCorr_UNRZ = zeros(1, total_samples);

sample = 1;
        for tau = 0:total_samples-1
            sample_values1 = Ensemble_UNRZ(:, sample);
            sample_values2 = Ensemble_UNRZ(:, sample+tau);
            squared_values = sample_values1 .* sample_values2;
            Stat_AutoCorr_UNRZ(tau + 1) = sum(squared_values) /
NUM_OF_SIGNALS;
        end

% Make it symmetric
Stat_AutoCorr_UNRZ_Flipped = fliplr(Stat_AutoCorr_UNRZ(2:end));
```

```
Stat_AutoCorr_Even_UNRZ = [Stat_AutoCorr_UNRZ_Flipped, Stat_AutoCorr_UNRZ];

% Define lag axis
tau_values = - (total_samples - 1) : (total_samples - 1);

% ======= Theoretical Statistical Autocorrelation ========

Tb_theory = samples_per_bit;  % Rename Tb for theoretical calculations
tau_theory_values = 0:(total_samples - 1);  % Consider only non-negative
lags

right_half_theory = zeros(1, total_samples);

for i = 1:total_samples
    tau_theory = tau_theory_values(i);  % Use the theoretical tau values
    if tau_theory >= Tb_theory
        right_half_theory(i) = A^2 / 4;
    else
        right_half_theory(i) = (A^2 / 2) * (1 - 0.5 * (tau_theory /
Tb_theory));
    end
end

% Mirror the right half to get the full theoretical autocorrelation
Theoretical_AutoCorr_UNRZ = [fliplr(right_half_theory(2:end)),
right_half_theory];

% Define theoretical tau axis
tau_theory_full = [-fliplr(tau_theory_values(2:end)), tau_theory_values];

% Plot Statistical and Theoretical Autocorrelation
figure;
subplot(3,1,1);
plot(tau_values, Stat_AutoCorr_Even_UNRZ, 'r', 'LineWidth', 1.5); hold on;
plot(tau_theory_full, Theoretical_AutoCorr_UNRZ, 'b--', 'LineWidth', 1.5);
legend('Simulated', 'Theoretical');
title('Statistical Autocorrelation of Unipolar NRZ');
xlabel('Lag (\tau)');
ylabel('Autocorrelation');
grid on;
xlim([-700 700]);  % Adjust x-axis range



%% ======= Statistical Autocorrelation For PNRZ ========
Stat_AutoCorr_PNRZ = zeros(1, total_samples);

sample = 1;
        for tau = 0:total_samples-1
            sample_values1 = Ensemble_PNRZ(:, sample);
            sample_values2 = Ensemble_PNRZ(:, sample+tau);
            squared_values = sample_values1 .* sample_values2;
            Stat_AutoCorr_PNRZ(tau + 1) = sum(squared_values) /
NUM_OF_SIGNALS;
        end

% Make it symmetric
Stat_AutoCorr_PNRZ_Flipped = fliplr(Stat_AutoCorr_PNRZ(2:end));
Stat_AutoCorr_Even_PNRZ = [Stat_AutoCorr_PNRZ_Flipped, Stat_AutoCorr_PNRZ];
```

```
% Define lag axis
tau_values = - (total_samples - 1) : (total_samples - 1);

% ======= Theoretical Statistical Autocorrelation ========

Tb_theory = samples_per_bit;  % Rename Tb for theoretical calculations
tau_theory_values = 0:(total_samples - 1);  % Consider only non-negative
lags

right_half_theory = zeros(1, total_samples);

for i = 1:total_samples
    tau_theory = tau_theory_values(i);  % Use the theoretical tau values
    if tau_theory >= Tb_theory
        right_half_theory(i) = 0;
    else
        right_half_theory(i) = A^2 * (1 - (tau_theory / Tb_theory));
    end
end

% Mirror the right half to get the full theoretical autocorrelation
Theoretical_AutoCorr_PNRZ = [fliplr(right_half_theory(2:end)),
right_half_theory];

% Define theoretical tau axis
tau_theory_full = [-fliplr(tau_theory_values(2:end)), tau_theory_values];


% Plot Statistical Autocorrelation
subplot(3,1,2);
plot(tau_values, Stat_AutoCorr_Even_PNRZ, 'b', 'LineWidth', 1.5); hold on;
plot(tau_theory_full, Theoretical_AutoCorr_PNRZ, 'g--', 'LineWidth', 1.5);
legend('Simulated', 'Theoretical');
title('Statistical Autocorrelation of Polar NRZ');
xlabel('Lag (\tau)');
ylabel('Autocorrelation');
grid on;
xlim([-700 700]);  % Adjust x-axis range



%% ======= Statistical Autocorrelation For PRZ ========
Stat_AutoCorr_PRZ = zeros(1, total_samples);


sample = 1;
        for tau = 0:total_samples-1
            sample_values1 = Ensemble_PRZ(:, sample);
            sample_values2 = Ensemble_PRZ(:, sample+tau);
            squared_values = sample_values1 .* sample_values2;
            Stat_AutoCorr_PRZ(tau + 1) = sum(squared_values) /
NUM_OF_SIGNALS;
        end


% Make it symmetric
Stat_AutoCorr_PRZ_Flipped = fliplr(Stat_AutoCorr_PRZ(2:end));
```

```
Stat_AutoCorr_Even_PRZ = [Stat_AutoCorr_PRZ_Flipped, Stat_AutoCorr_PRZ];

% Define lag axis
tau_values = - (total_samples - 1) : (total_samples - 1);

% ======= Theoretical Statistical Autocorrelation ========

Tb_theory = samples_per_bit;  % Rename Tb for theoretical calculations
tau_theory_values = 0:(total_samples - 1);  % Consider only non-negative
lags

right_half_theory = zeros(1, total_samples);

for i = 1:total_samples
    tau_theory = tau_theory_values(i);  % Use the theoretical tau values
    if tau_theory >= Tb_theory/2
        right_half_theory(i) = 0;
    else
        right_half_theory(i) = (A^2/2) * (1-(2*tau_theory / Tb_theory));
    end
end

% Mirror the right half to get the full theoretical autocorrelation
Theoretical_AutoCorr_PRZ = [fliplr(right_half_theory(2:end)),
right_half_theory];

% Define theoretical tau axis
tau_theory_full = [-fliplr(tau_theory_values(2:end)), tau_theory_values];

% Plot Statistical Autocorrelation
subplot(3,1,3);
plot(tau_values, Stat_AutoCorr_Even_PRZ, 'g', 'LineWidth', 1.5); hold on;
plot(tau_theory_full, Theoretical_AutoCorr_PRZ, 'r--', 'LineWidth', 1.5);
legend('Simulated', 'Theoretical');
title('Statistical Autocorrelation of Polar RZ');
xlabel('Lag (\tau)');
ylabel('Autocorrelation');
grid on;
xlim([-700 700]);  % Adjust x-axis range

%% ======= Time Autocorrelation For UNRZ ========
x_t = Ensemble_UNRZ(1, :);
Time_AutoCorr_UNRZ = zeros(1, total_samples);

for tau = 0:total_samples-1
    shifted_x = circshift(x_t, [0, tau]);
    sum_corr = sum(x_t .* shifted_x);

    % Average over time
    Time_AutoCorr_UNRZ(tau + 1) = sum_corr / total_samples;
end

% Make it symmetric
Time_AutoCorr_UNRZ_Flipped = fliplr(Time_AutoCorr_UNRZ(2:end));
Time_AutoCorr_Even_UNRZ = [Time_AutoCorr_UNRZ_Flipped, Time_AutoCorr_UNRZ];

% Plot Time Autocorrelation
figure;
subplot(3,1,1);
```

```
plot(tau_values, Time_AutoCorr_Even_UNRZ,'r', 'LineWidth', 1.5);
title('Time Autocorrelation of Unipolar NRZ');
xlabel('Lag (\tau)');
ylabel('Autocorrelation');
grid on;
xlim([-700 700]);  % Adjust x-axis range

%% ======= Time Autocorrelation For PNRZ ========
x_t = Ensemble_PNRZ(1, :);
Time_AutoCorr_PNRZ = zeros(1, total_samples);

for tau = 0:total_samples-1
    shifted_x = circshift(x_t, [0, tau]);
    sum_corr = sum(x_t .* shifted_x);

    % Average over time
    Time_AutoCorr_PNRZ(tau + 1) = sum_corr / total_samples;
end

% Make it symmetric
Time_AutoCorr_PNRZ_Flipped = fliplr(Time_AutoCorr_PNRZ(2:end));
Time_AutoCorr_Even_PNRZ = [Time_AutoCorr_PNRZ_Flipped, Time_AutoCorr_PNRZ];

% Plot Time Autocorrelation
subplot(3,1,2);
plot(tau_values, Time_AutoCorr_Even_PNRZ,'g', 'LineWidth', 1.5);
title('Time Autocorrelation of polar NRZ');
xlabel('Lag (\tau)');
ylabel('Autocorrelation');
grid on;
xlim([-700 700]);  % Adjust x-axis range

%% ======= Time Autocorrelation For PRZ ========
x_t = Ensemble_PRZ(1, :);
Time_AutoCorr_PRZ = zeros(1, total_samples);

for tau = 0:total_samples-1
    shifted_x = circshift(x_t, [0, tau]);
    sum_corr = sum(x_t .* shifted_x);

    % Average over time
    Time_AutoCorr_PRZ(tau + 1) = sum_corr / total_samples;
end

% Make it symmetric
Time_AutoCorr_PRZ_Flipped = fliplr(Time_AutoCorr_PRZ(2:end));
Time_AutoCorr_Even_PRZ = [Time_AutoCorr_PRZ_Flipped, Time_AutoCorr_PRZ];

% Plot Time Autocorrelation
subplot(3,1,3);
plot(tau_values, Time_AutoCorr_Even_PRZ,'b', 'LineWidth', 1.5);
title('Time Autocorrelation of polar RZ');
xlabel('Lag (\tau)');
ylabel('Autocorrelation');
grid on;
xlim([-700 700]);  % Adjust x-axis range
%% ======= PSD For UNRZ ========
```

```
% Ensure FFT matches the length of the autocorrelation function
N = length(Stat_AutoCorr_Even_UNRZ);

% Define the frequency axis for the Fourier Transform (centered at 0 Hz)
freq_axis = linspace(-fs/2, fs/2, N);

% PSD is the Fourier Transform (FFT) of the autocorrelation function
PSD_Uni_Polar = abs(fftshift(fft(Stat_AutoCorr_Even_UNRZ,N))); % Unipolar
NRZ

theoretical_PSD_Uni_Polar = fs* (A^2 * Tb_theory / 4) * (sinc(freq_axis *
Tb_theory).^2);
figure;
subplot(3,1,1);
plot(freq_axis, F_AutoCorr_simulated_UNRZ, 'g');
hold on;
plot(freq_axis, abs(theoretical_PSD_Uni_Polar) * A^2 * Tb_theory * 2, 'r--
');
legend('Simulated', 'Theoretical');
xlabel('Frequency (Hz)');
ylabel('PSD');
title('PSD of UNRZ');
xlim([-100 100]);

%% ======================PSD for PNRZ============================
PSD_UNRZ = abs(fftshift(fft(Stat_AutoCorr_UNRZ)));


F_AutoCorr_theoritical_PNRZ=fftshift(fft(Theoretical_AutoCorr_PNRZ))*1/fs;


% Ensure FFT matches the length of the autocorrelation function
N = length(Stat_AutoCorr_Even_PNRZ);
% Define the frequency axis for the Fourier Transform (centered at 0 Hz)
freq_axis = linspace(-fs/2, fs/2, N);

PSD_Polar_NRZ = abs(fftshift(fft(Stat_AutoCorr_Even_PNRZ, N))); % Polar NRZ


subplot(3,1,2);
plot(freq_axis, PSD_Polar_NRZ, 'g');
hold on;
plot(freq_axis, abs(F_AutoCorr_theoritical_PNRZ) * A^2 * Tb_theory * 2, 'r-
-');
legend('Simulated', 'Theoretical');
xlabel('Frequency (Hz)');
ylabel('PSD');
title('PSD of PNRZ');
xlim([-100 100]);



%% ======= PSD For PRZ ========

F_AutoCorr_theoritical_PRZ=fftshift(fft(Theoretical_AutoCorr_PRZ))*1/fs;


% Ensure FFT matches the length of the autocorrelation function
N = length(Stat_AutoCorr_Even_PRZ);
```

```
% Define the frequency axis for the Fourier Transform (centered at 0 Hz)
freq_axis = linspace(-fs/2, fs/2, N);

% PSD is the Fourier Transform (FFT) of the autocorrelation function
PSD_Polar_RZ = abs(fftshift(fft(Stat_AutoCorr_Even_PRZ, N))); % Polar RZ

subplot(3,1,3);
plot(freq_axis, PSD_Polar_RZ, 'g');
hold on;
plot(freq_axis, abs(F_AutoCorr_theoritical_PRZ) * A^2 * Tb_theory * 2, 'r--
');
legend('Simulated', 'Theoretical');
xlabel('Frequency (Hz)');
ylabel('PSD');
title('PSD of PRZ');
xlim([-100 100]);
```

References

[1] Dr. A. Nafie, *Lecture 4 Slide 23.*